

Hochschule
Zittau/Görlitz
UNIVERSITY OF APPLIED SCIENCES

# Polling & Long Polling

## Web Engineering 1 (319300)

Vorlesung 04

M.Sc. Christopher-Manuel Hilgner

Hochschule Zittau/Görlitz

christopher.hilgner@stud.hszg.de

2026-10-27

# Agenda

- Polling & Long Polling
- Web Sockets

# Polling & Long Polling

# Das Problem

- HTTP ist ein **Request-Response-Protokoll**: Der Client fragt, der Server antwortet
- Der Server kann von sich aus **nichts** an den Browser schicken
- Frage: Wie erfährt die Seite von neuen Daten (Chatnachricht, Benachrichtigung, Auftragsstatus)?

Drei Ansätze mit steigender Komplexität:

| Ansatz | Idee |
|---|---|
| Polling | Client fragt in festen Abständen nach |
| Long Polling | Server hält die Anfrage offen, bis es etwas Neues gibt |
| Server-Sent Events, Web Sockets | Dauerhafte Verbindung (kommt später) |

# Polling

- Der Client sendet in regelmäßigen Abständen eine Anfrage, egal ob es etwas Neues gibt
- Einfachste Variante
- Funktioniert mit jedem Server und jedem Proxy

# Polling

## Beispiel

JavaScript

```
async function pollStatus() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  }
}
setInterval(pollStatus, 5000); // alle 5 Sekunden
```

# Polling Umsetzung

Problem von `setInterval`: Dauert eine Antwort länger als das Intervall, überlappen sich beide Anfragen.

**Besser**: rekursives `setTimeout`. Die nächste Anfrage startet erst, wenn die letzte fertig ist.

**Weitere Verbesserungen**:

- Polling stoppen, wenn der Tab im Hintergrund ist (`document.visibilityState`, Event `visibilitychange`)
- Bei Fehlern das Intervall vergrößern (Exponential Backoff: 5s, 10s, 20s, …)
- Abbruch über `AbortController`, z.B. beim Verlassen der Seite
- Serverseitig `ETag`/`If-None-Match` nutzen: Antwort `304 Not Modified` spart Daten

# Polling Umsetzung

JS JavaScript

```
async function poll() {
  try {
    const response = await fetch("/api/orders/42/status");
    if (!response.ok) throw new Error("HTTP ${response.status}");
    const { status } = await response.json();
    updateUI(status);
  } catch (error) {
    console.error(error);
  } finally {
    setTimeout(poll, 5000);
  }
}
poll();
```

# Bewertung

**Vorteile**:

- Sehr einfach zu implementieren
- Funktioniert überall (Proxys, Firewalls, Caches)
- Server bleibt zustandslos

**Nachteile**

- Latenz bis zu einem Intervall (bei 5s im Schnitt 2.5s Verzögerung)
- Viele unnötige Requests, wenn sich selten etwas ändert
- Kurzes Intervall = Extra Last auf Server und Netz
- Langes Intervall = Träge UI

**Typischer Einsatz**: Dashboards, Statusabfragen von langlaufenden Jobs, wenn Verzögerungen von Sekunden egal sind

# Long Polling

# Long Polling

**Idee**: Der Server beantwortet die Anfrage nicht sofort, sondern hält sie offen, bis es etwas Neues gibt (oder ein Timeout erreicht ist). Danach öffnet der Client sofort die nächste Anfrage.

## Ablauf

1. Client sendet Anfrage
2. Server wartet (z.B. bis zu 30s)
3. Neue Daten da: Server antwortet sofort. Timeout: Server antwortet leer (z.B. 204 No Content)
4. Client verarbeitet die Antwort und startet direkt die nächste Anfrage

# Long Polling

JS JavaScript

```
async function longPoll(lastId = 0) {
  while (true) {
    try {
      const response = await fetch("/path?after=${lastId}");
      if (response.status == 200) {
        const messages = await response.json();
        messages.forEach(show);
        lastId = messages.at(-1).id;
      }
      // 204 = Timeout ohne Neuigkeiten, erneut fragen
    } catch (error) {
      console.error(error);
      await new Promise((resolve) => setTimeout(resolve,
      5000)); // Pause bei Fehler
```

# Long Polling

```
14        }
15      }
16    }
17    longPoll();
```

**Wichtig**: Der Client merkt sich, was er schon hat (`lastId`), damit keine Nachricht zwischen zwei Anfragen verloren geht.

# Long Polling bewertet

**Vorteile**:

- Fast sofortige Zustellung
- Weniger leere Requests als Polling
- Läuft über normales HTTP

**Nachteile**:

- Der Server hält viele offene Verbindungen
- Proxys und Load Balancer beenden lange Verbindungen (Timeout oft 30 bis 60s)
- Pro Domain gibt es Verbindungslimits (HTTP/1.1: meist 6 pro Browser)
- Zwischen zwei Anfragen entsteht kurz eine „Lücke“

# Long Polling bewertet

- Server-Timeout kürzer als der Proxy-Timeout wählen
- Server muss viele wartende Anfragen effizient halten können (asynchrones/nicht blockierendes I/O, bei Thread-pro-Request-Modellen wird das sehr schnell eng)
- Bei Fehlern immer mit Pause und Backoff neu verbinden, sonst entsteht eine „Thundering Herd“, wenn der Server wieder erreichbar ist

# Vergleich und Ausblick

| | Polling | Long Poll. | SSE | WebSocket |
|---|---|---|---|---|
| Richtung | Client → Server | Client → Server | Server → Client | beidseitig |
| Latenz | Intervall | gering | gering | gering |
| Verbindung | neu pro Anfrage | neu pro Antwort | dauerhaft | dauerhaft |
| Protokoll | HTTP | HTTP | HTTP (`EventSource`) | eigenes (`ws://`, `wss://`) |
| Aufwand | sehr gering | mittel | gering | höher |

# Vergleich und Ausblick

**Faustregel**:

- Selten ändernde Daten, Sekunden egal: **Polling**
- Neuigkeiten sollen schnell ankommen, nur Server → Client: **SSE** (oder Long Polling als Fallback)
- Echte Zwei-Wege-Kommunikation (Chat, Spiele, Kollaboration): **WebSocket**

Kurzer Ausblick auf SSE:

JavaScript

```
1  const source = new EventSource("/api/events");
2  source.onmessage = (event) =>
   console.log(JSON.parse(event.data));
```

# Typische Fehler

- `setInterval` mit `async`-Funktion: Anfragen überlappen sich, wenn der Server langsam ist
- **Kein Abbruch**: Polling läuft weiter, obwohl die Seite oder Komponente längst weg ist (Memory Leak, vgl. Folie zu Garbage Collection)
- **Zu kurze Intervalle**: 1-Sekunden-Polling von 1000 Nutzern sind 1000 Requests pro Sekunde
- **Endlosschleife ohne Pause bei Fehlern**: Der Client hämmert dann den Server mit Fehlversuchen zu

# Web Sockets

# Web Sockets